INT234 - Predictive Analytics

PROJECT REPORT

(July–December 2026)

River Water Quality in India

Submitted by

Palla Guru Tharun

Registration No : 12404241

Programme and Section : BTECH CSE (K3P24FV)

Course Code : INT234

Under the Guidance of

(**Dr. Madhu Bala UID: 31770**)

Discipline of CSE/IT

School of Computer Science & Engineering

Lovely Professional University, Phagwara, Punjab.

DECLARATION

I, Palla Guru Tharun, student of BTECH CSE under CSE/IT Discipline at Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Date: 31-10-2026 Signature

Registration No : 12404241 Palla Guru Tharun

CERTIFICATE

This is to certify that Palla Guru Tharun bearing Registration no. 12404241 has completed the INT234 project titled, “River Water Quality in India” under my guidance and supervision. To the best of my knowledge, the present work is the result of his original development, effort and study.

Signature and Name of the Supervisor: Dr. Madhu Bala (31770)

School of Computer Science & Engineering

Lovely Professional University

Phagwara, Punjab.

Date: 31-10-2026

ACKNOWLEDGEMENT

I would like to thank my project supervisor, Dr. Madhu Bala, for the guidance and support given during this project. I am also thankful to the School of Computer Science & Engineering and Lovely Professional University for giving me the chance to work on a practical data science project.

TABLE OF CONTENTS

1. INTRODUCTION 1
1.1 Background of the Study 1
1.2 Introduction to River Water Quality 1
1.3 Role of Predictive Analytics in Quality Analysis 1
1.4 Problem Statement 2
1.5 Project Overview 2
1.6 Project Objectives 2
1.7 Project Scope 2
1.8 Technologies and Libraries Used 3
1.9 Significance of the Study 3
1.10 Organization of the Report 3
2. SOURCE OF DATASET 4
2.1 Introduction to the Dataset 4
2.2 River Water Quality Across Different States of India 4
2.3 Dataset Description 4
2.4 Dataset Dimensions 5
2.5 Important Variables Used 5
2.6 Target Variables 5
2.7 Dataset Characteristics 6
2.8 Challenges in Working with the Dataset 6
3. EDA PROCESS 7
3.1 Introduction to Exploratory Data Analysis 7
3.2 Importing Required Libraries 7
3.3 Loading the Dataset 7
3.4 Dataset Inspection 7
3.5 Dataset Shape and Structure 7
3.6 Data Types 8
3.7 Descriptive Statistics 8
3.8 Missing Value Analysis 9
3.9 Duplicate Record Analysis 9
3.10 Data Cleaning 9
3.11 Outlier Detection Using IQR 10
3.12 Numeric Columns Analysis 11
3.13 Correlation Analysis 12
3.14 Summary of EDA Findings 13
4. ANALYSIS ON DATASET 14
4.1 Multiple Linear Regression 14
4.1.1 Introduction 14
4.1.2 General Description 14
4.1.3 Specific Requirements, Functions and Formulas 14
4.1.4 Analysis Results 15
4.1.5 Visualization 17
4.2 Polynomial Regression (Degree = 2) 17
4.2.1 Introduction 17
4.2.2 General Description 18

4.2.3 Specific Requirements, Functions and Formulas 18
4.2.4 Analysis Results 19
4.2.5 Visualization 19
4.3 Logistic Regression 20
4.3.1 Introduction 20
4.3.2 General Description 20
4.3.3 Specific Requirements, Functions and Formulas 21
4.3.4 Analysis Results 21
4.3.5 Visualization 22
4.4 K-Nearest Neighbors (KNN) Classification 23
4.4.1 Introduction 23
4.4.2 General Description 23
4.4.3 Specific Requirements, Functions and Formulas 24
4.4.4 Analysis Results 24
4.4.5 Visualization 25
4.5 Comparative Analysis of Regression Models 26
4.6 Comparative Analysis of Classification Models 26
4.7 Overall Model Performance Discussion 27
5. CONCLUSION 28
5.1 Project Summary 28
5.2 Major Activities and Findings 28
5.3 Model Results 28
5.4 Challenges and Learning 29
5.5 Overall Outcome 29
6. FUTURE SCOPE 30
6.1 Completed Work and Proposed Work 30
6.2 Modelling Improvements 30
6.3 Better Handling of Missing Data and Outliers 30
6.4 Classification Targets and Metrics 30
6.5 Features and Data 30
6.6 Application Development 31
6.7 Limitations of the Present Work 31
7. REFERENCES 32

LIST OF FIGURES

Figure 3.1: Average Maximum BOD Across Indian States 11
Figure 3.2: Boxplot of all eighteen water-quality columns (outlier identification) 12
Figure 3.3: Correlation Heatmap of River Water Quality 12
Figure 4.1: Actual vs Predicted Values – Multiple Linear Regression 17
Figure 4.2: Actual vs Predicted Values – Polynomial Regression (degree 2) 20
Figure 4.3: Confusion Matrix – Logistic Regression 23
Figure 4.4: Confusion Matrix – KNN Classifier 26

LIST OF TABLES

Table 1.1: Technologies and libraries used in the project 3
Table 2.1: Column groups of the dataset 4
Table 2.2: Dataset dimensions at each stage 5
Table 2.3: Input features and target variable of each model 5
Table 2.4: Class labels created for the classification tasks 6
Table 3.1: Distribution of column data types 8
Table 3.2: Descriptive statistics of the water-quality columns (after filling, before outlier removal) 8
Table 3.3: Missing values in the measurement columns before cleaning 9
Table 3.4: Data cleaning results 10
Table 3.5: Outlier detection results using the IQR method 10
Table 3.6: Summary of EDA findings and their effect on modelling 13
Table 4.1: Regression metrics and how to read them 15
Table 4.2: Evaluation metrics of Multiple Linear Regression (target: max_bod) 15
Table 4.3: Coefficients of the Multiple Linear Regression model 16
Table 4.4: Actual and predicted max_bod for the first eight test records (MLR) 17
Table 4.5: Evaluation metrics of Polynomial Regression, degree 2 (target: max_temp) 19
Table 4.6: First eight test records: min_temp, actual and predicted max_temp 19
Table 4.7: Confusion matrix of Logistic Regression (max_fecal) 21
Table 4.8: Class-wise precision, recall and F1-score of Logistic Regression (calculated from Table 4.7) 22
Table 4.9: Coefficients of the Logistic Regression model on scaled inputs 22
Table 4.10: Confusion matrix of the KNN classifier (max_do) 24
Table 4.11: Class-wise precision, recall and F1-score of KNN (calculated from Table 4.10) 25
Table 4.12: Logistic Regression and KNN: approach and result 25
Table 4.13: Comparison of regression models 26
Table 4.14: Comparison of classification models 26
Table 4.15: Overall comparison of the four models 27
Table 5.1: Summary of key results 28
Table 5.2: Objectives and outcomes 29
Table 6.1: Limitations of this project and proposed improvements 31

1. INTRODUCTION

1.1 Background of the Study

Rivers provide a main source of drinking water, irrigation, fisheries, and industrial water in India, and they also receive treated and untreated wastewater from cities, farms, and industries. As the status of the river varies according to location and time of year, the monitoring organizations measure a suite of physical, chemical, and biological parameters at fixed stations on the river. In many years, such measurements lead to a huge data set of the behavior of river water in various conditions and locations. This project utilizes such a data set in which river monitoring stations in India have been monitored throughout the calendar years 2012 to 2023.

1.2 Introduction to River Water Quality

River waters are one of the chief sources of water for drinking purposes, irrigation purposes, fisheries, and industrial uses in India. The river waters are also subjected to receiving the effluents from the urban areas, agricultural operations, and industries. Because the status of the river changes from place to place and season to season, the monitoring agencies undertake the task of recording the values of several parameters concerning the river waters at the fixed stations of the rivers. For many years, such observations generate a large amount of data regarding the characteristics of the river waters in various conditions and places. This project relies on the data generated through this process in India between the calendar years 2012 to 2023.

In this dataset, two columns are of particular importance. The maximum BOD (max_bod) is chosen to be the numeric target, as BOD is commonly used to measure the organic load in water. The maximum fecal coliform (max_fecal) and the maximum dissolved oxygen (max_do) are classified into high and low classes and are used for classification targets. The maximum temperature (max_temp) is chosen to be the target of the polynomial model.

1.3 Role of Predictive Analytics in Quality Analysis

In predictive analytics, data where the answers are already known are used to discover a rule that will predict the answer for new data. The prediction in case where the answer will be a number, like the maximum BOD, is called regression. The prediction of a category as “high”

or “low” fecal coliform, on the other hand, is called classification. Both of them fall under supervised learning. These types of models, when applied in water quality analysis, will be able to determine correlations between parameters, provide a rough estimate for an unavailable or expensive parameter and assist in comparing stations. In this project, the models have been created to demonstrate how they work; they would require extensive validation before being applied to actual decision making regarding river management.

1.4 Problem Statement

Using river water quality data, this project sets out to solve four prediction problems using four different machine learning techniques, which are (a) predicting the maximum Biological Oxygen Demand from nine other water quality metrics using Multiple Linear Regression; (b) predicting the maximum temperature from the minimum temperature using Polynomial Regression with degree 2; (c) classification of the maximum fecal coliform level being high or low, with twelve metrics and logistic regression, and (d) classification of the maximum level of dissolved oxygen being high or low with eleven metrics and k-nearest neighbors (K = 5). The data contains several gaps, extreme values, and also a very wide range of scales in the measurement data.

1.5 Project Overview

Utilizing the river water quality data, the purpose of the study is to address four different prediction tasks by applying four separate models: (a) predicting the maximum BOD of the record from nine other water quality measurements utilizing Multiple Linear Regression; (b) predicting the maximum temperature from the minimum temperature utilizing Polynomial Regression of degree 2; (c) classifying whether the maximum fecal coliform of the record is high or low from twelve measurements using Logistic Regression; and (d) classifying whether the maximum dissolved oxygen of the record is high or low from eleven measurements using K-Nearest Neighbors with K = 5. The data is highly incomplete with numerous outliers and wide-ranging scales.

1.6 Project Objectives

1. To load and inspect the river water-quality dataset of 11,721 rows and 23 columns.
2. To handle missing values and to detect and remove outliers using the IQR method.
3. To explore the data through a bar chart, a correlation heatmap and a boxplot.

4. To predict maximum BOD using Multiple Linear Regression and maximum temperature using Polynomial Regression (degree 2).
5. To classify maximum fecal coliform using Logistic Regression and maximum dissolved oxygen using K-Nearest Neighbors.
6. To evaluate every model with suitable metrics and to compare the results honestly.

1.7 Project Scope

The project includes data exploration, missing value treatment, outlier detection and treatment, visualization, modeling and evaluation for the four models mentioned above, with one random split into train and test datasets. Hyperparameter tuning, cross validation, other methods, time series analysis of the years, or deployment are not part of the project. The class labels for the two classification problems are formed by splitting the target variable at the median; therefore, no water quality standards are used in this project.

1.8 Technologies and Libraries Used

Table 1.1: Technologies and libraries used in the project

Tool / Library	Purpose in the project
Python	Programming language used to write the whole analysis
Pandas	Loading the CSV file, cleaning column names, grouping by state, filling missing values, filtering rows
NumPy	Numerical operations, such as the square root used for RMSE
Matplotlib	Boxplot and the scatter plots of actual and predicted values
Seaborn	Bar chart of average BOD, correlation heatmap and confusion-matrix heatmaps
Scikit-learn	Train-test split, regression and classification models, scaling, polynomial features and metrics

1.9 Significance of the Study

Monitoring of rivers results in obtaining numerous measurements, however, they are not always full, and the indicators themselves are interrelated in a complex way. Analysis of correlation between indicators, and ability of estimation of one indicator using the others, is a good start for more detailed analysis. Estimation of maximum BOD through other measurements is an example of how much of the organic-load indicator depends on the oxygen, pH, conductivity, and nitrate measurements. Division of fecal coliform and dissolved oxygen into groups 'high' and 'low' shows if it is possible to use simple models to distinguish clean measurements from polluted ones. From the viewpoint of a student, such task is also

interesting as it gives an example of the whole process from getting a messy public dataset to getting evaluated models.

1.10 Organization of the Report

The report follows the structure prescribed by the university. Chapter 1 introduces the study. Chapter 2 describes the source and structure of the dataset. Chapter 3 presents the exploratory data analysis, including cleaning and outlier handling. Chapter 4 explains the four models, with the five mandatory parts for each, and compares them. Chapter 5 gives the conclusion, Chapter 6 suggests future work, and Chapter 7 lists the references.

2. SOURCE OF DATASET

2.1 Introduction to the Dataset

The data used in this project is stored in the file. Each row describes the water-quality record of one monitoring location for one calendar-year label, and each column holds one identifying item or one measured value. The file was read into a Pandas DataFrame, and all analysis in this report is based on it.

2.2 River Water Quality Across Different States of India

The file covers river monitoring locations in 32 states and union territories of India, identified by 1,804 different station codes. The Year column holds text labels of the form “Calendar Year (Jan - Dec), 2023”. Eleven different labels occur, ranging from 2012 to 2023; the label for 2015 does not appear in the file. The source organisation, the download page and the sampling procedure of the data are not stated in the supplied materials, so this report does not describe them. The origin of the file should be confirmed and cited by the author before submission.

2.3 Dataset Description

The file has 23 columns. Five of them describe the record: Country, Year, State/Ut Name, Station Code and Monitoring Location. The remaining eighteen are numeric measurements that come in nine minimum–maximum pairs. Table 2.1 groups the columns. A trailing space in the name of the `min_cond` column was removed by the notebook using `df.columns.str.strip()`.

Table 2.1: Column groups of the dataset

Group	Columns	Names
Identification / text	4 text + 1 numeric	Country, Year, State/Ut Name, Monitoring Location; Station Code (numeric code)
Temperature	2	min_temp, max_temp
Dissolved oxygen	2	min_do, max_do
pH	2	min_ph, max_ph
Conductivity	2	min_cond, max_cond
Biochemical oxygen demand	2	min_bod, max_bod
Nitrate	2	min_nitrate, max_nitrate
Fecal coliform	2	min_fecal, max_fecal
Total coliform	2	min_tc, max_tc
Fecal streptococci	2	min_fs, max_fs

2.4 Dataset Dimensions

Table 2.2 shows the size of the data at each stage. Filling the missing values does not change the number of rows. The outlier removal step of Section 3.11 reduces the data to 4,051 rows, which is about 34.6% of the original rows; this smaller data frame is the one used by all four models.

Table 2.2: Dataset dimensions at each stage

Stage	Rows	Columns	Rows removed
Original dataset	11,721	23	-
After filling missing values	11,721	23	0
After outlier removal (IQR)	4,051	23	7,670 (65.4%)

2.5 Important Variables Used

Different models use different subsets of the eighteen numeric columns, because the target column of each task must not appear among its own inputs. Table 2.3 lists the inputs and the target of each model. The variable names are taken directly from the code of `project.py`.

Table 2.3: Input features and target variable of each model

Model	Target	No. of inputs	Input features
Multiple Linear Regression	max_bod	9	min_do, max_do, min_ph, max_ph, min_cond, max_cond, min_bod, min_nitrate, max_nitrate
Polynomial Regression (degree 2)	max_temp	1	min_temp
Logistic Regression	max_fecal (high / low)	12	min_temp, max_temp, min_do, max_do, min_ph, max_ph, min_cond, max_cond, min_bod, max_bod, min_nitrate, max_nitrate
KNN (K = 5)	max_do (high / low)	11	min_temp, max_temp, min_ph, max_ph, min_cond, max_cond, min_bod, max_bod, min_nitrate, max_nitrate, max_fecal

The selection follows a simple idea. For BOD, the other oxygen, acidity, conductivity and nutrient measurements, together with the minimum BOD of the same record, are used as likely companions. For the fecal coliform task, the physical and chemical measurements are used as inputs, and the coliform, total coliform and streptococci columns are left out. For the dissolved oxygen task, the maximum fecal coliform is added to the physical and chemical measurements. These choices were made by the author in the notebook; the report does not claim that they are the best possible choices.

2.6 Target Variables

`max_bod` and `max_temp` are continuous numbers, so predicting them is a regression task. `max_fecal` and `max_do` are also continuous, but the notebook converts each of them into a two-class label using the median of the cleaned data: a record gets class 1 if its value is greater than or equal to the median and class 0 otherwise. Table 2.4 gives the thresholds. Because the median is used, the two classes are close to equal in size, which is an important difference from an imbalanced classification problem.

Table 2.4: Class labels created for the classification tasks

Target column	Median threshold	Share of class 1	Rule
max_fecal	310	50.04%	class 1 if max_fecal ≥ 310
max_do	8.0	52.78%	class 1 if max_do ≥ 8.0

2.7 Dataset Characteristics

- 11,721 rows and 23 columns in the original file.
- 19 columns of type float and 4 columns of text type.
- 30,278 missing values in total, of which 30,275 lie in the eighteen numeric measurement columns and 3 in Station Code.
- No duplicate rows in the original file.
- Very different scales: for example max_ph has a median near 8 but a maximum of 3,334, while max_fecal has a median of 910 and a maximum of 20 billion.

2.8 Challenges in Working with the Dataset

The main difficulty is missing data. The fecal streptococci columns (`min_fs`, `max_fs`) are missing in about 77% of the rows, and `min_nitrate` is missing in about 18%. A second difficulty is the presence of extreme and sometimes impossible values. A pH value of 3,334 lies far outside the usual pH scale, which suggests a recording or entry error, and several coliform values run into billions. A third difficulty is the large difference in scale between columns, which matters for the distance-based KNN model and for the interpretation of regression coefficients. Finally, the notebook handles all of these problems with simple, general steps (mean filling and IQR removal), which keep the workflow easy to follow but have side effects that are discussed next page.

3. EDA PROCESS

3.1 Introduction to Exploratory Data Analysis

Exploratory Data Analysis (EDA) means studying the data carefully before building models. In this project EDA was used to check the size and type of the data, to count missing values and duplicates, to fill the gaps, to find and remove outliers, and to look at the shape of the data and the relations between the columns.

3.2 Importing Required Libraries

The notebook begins by importing the libraries listed below. Pandas and NumPy handle data and numbers, Matplotlib and Seaborn draw the charts, and Scikit-learn provides the models and the metrics.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear model import LinearRegression, LogisticRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
from sklearn.metrics import accuracy_score, confusion_matrix
```

3.3 Loading the Dataset

The data was loaded with `pd.read_csv()`, and the column names were cleaned with `df.columns = df.columns.str.strip()`. This step is needed because the original header contains the names `min_cond` and `max_cond` with a trailing space on the first of them, which would otherwise cause a column-not-found error when the name is used later. The first rows were then printed with `df.head()` to confirm that the file was read correctly.

3.4 Dataset Inspection

The notebook inspects the data with `df.head()`, `df.columns` and `df.shape`. These calls show the column names and confirm the number of rows and columns. The names of the eighteen measurement columns are then stored in Python variables (for example `min_temp = 'min_temp'`) and collected in the list `water_cols`, which is used in every later step.

3.5 Dataset Shape and Structure

The original dataset has 11,721 rows and 23 columns (Table 2.2). Each row is one monitoring record. The columns Country, Year, State/Ut Name and Monitoring Location hold text, Station Code is numeric, and the eighteen water-quality columns are numeric.

3.6 Data Types

Table 3.1: Distribution of column data types

Data type	Number of columns	Columns
float64	19	Station Code and the 18 measurements
Text (object / string)	4	Country, Year, State/Ut Name, Monitoring Location
Total	23	

Before further use, the eighteen measurement columns were passed through `pd.to_numeric(errors="coerce")`. This command turns any entry that cannot be read as a number into a missing value instead of raising an error. In this file all the measurement columns were already numeric, so the command acts as a safety step.

3.7 Descriptive Statistics

Table 3.2 summarises the eighteen measurement columns after the missing values were filled (Section 3.10) and before the outliers were removed. The notebook itself does not print `describe()`; the summary was computed from the same data to support the discussion. The table shows how large the differences in scale are. The mean of `max_fecal` is far above its median, and the mean of `max_cond` is more than twice its median, which means that a small number of very large values pull the mean upwards. Statistics for `min_fs` and `max_fs` are dominated by the filled mean value, because most of their rows were missing.

Table 3.2: Descriptive statistics of the water-quality columns (after filling, before outlier removal)

Column	Mean	Median	Minimum	Maximum
min_temp	18.5	19	0	35
max_temp	28.56	29	0	269
min_do	5.66	6	0	85
max_do	8.05	8	0	95
min_ph	7.93	7.2	0	2,499
max_ph	9.22	8.2	2.8	3,334
min_cond	283.24	213	0	36,100
max_cond	1,300.58	492.6	0	5.13e+5

Column	Mean	Median	Minimum	Maximum
min_bod	2.41	1.4	0	240
max_bod	6.8	2.8	0	675
min_nitrate	2.39	0.32	0	14,000
max_nitrate	33.29	1.6	0	1.70e+5
min_fecal	3,307.88	110	0	3.30e+6
max_fecal	6.62e+6	910	0	2.00e+10
min_tc	8,776.56	350	0	9.40e+6
max_tc	7.98e+6	1,800	0	2.00e+10
min_fs	1,276.88	1,276.88	1.8	2.10e+6
max_fs	57,640.77	57,640.77	-2	7.80e+7

3.8 Missing Value Analysis

The check `df.isnull().sum()` found 30,278 missing values in the original file. Table 3.3 lists the count for each measurement column. The only other column with missing data is Station Code, which has 3. The `min_fs` and `max_fs` columns are the most incomplete, each with almost 9,000 missing entries, which is about 77% of all rows.

Table 3.3: Missing values in the measurement columns before cleaning

Column	Missing	% of rows	Column	Missing	% of rows
min_temp	267	2.3%	max_bod	348	3.0%
max_temp	266	2.3%	min_nitrate	2086	17.8%
min_do	152	1.3%	max_nitrate	1167	10.0%
max_do	119	1.0%	min_fecal	1717	14.6%
min_ph	51	0.4%	max_fecal	1717	14.6%
max_ph	51	0.4%	min_tc	1347	11.5%
min_cond	494	4.2%	max_tc	1340	11.4%
max_cond	491	4.2%	min_fs	8985	76.7%
min_bod	693	5.9%	max_fs	8984	76.6%

3.9 Duplicate Record Analysis

The duplicate check returned 0, so the original file has no repeated rows and none had to be removed.

3.10 Data Cleaning

The notebook handles missing values in two lines of code:

```
df[water_cols] = df[water_cols].apply(pd.to_numeric, errors="coerce")
df[water_cols] = df[water_cols].fillna(df[water_cols].mean())
```

The first line converts the columns to numbers. The second replaces every missing entry in each column by the mean of that column. After this step, the number of missing values in the

measurement columns is 0. Mean filling is simple and keeps all 11,721 rows, but it has two effects that the reader should keep in mind. First, the filled value is the same for every missing entry of a column, so the spread of the column becomes smaller. Second, the mean is itself pulled up by extreme values, so the filled value can be much larger than a typical record. For `min_fs` and `max_fs` the effect is very strong, because about 77% of the entries are replaced by the same number (1,276.9 and 57,640.8 respectively).

Table 3.4: Data cleaning results

Check	Before	After	Change
Rows	11,721	11,721	none (filling step)
Columns	23	23	none
Missing values (18 measurement columns)	30,275	0	all filled with column mean
Duplicate rows	0	0	none

3.11 Outlier Detection Using IQR

The Interquartile Range (IQR) method marks a value as an outlier when it is below Q1 − 1.5 × IQR or above Q3 + 1.5 × IQR, where Q1 and Q3 are the first and third quartiles and IQR = Q3 − Q1. The notebook applied this rule to all eighteen measurement columns and counted the outliers in each column. Table 3.5 gives the bounds and the counts obtained by running the notebook code.

$$\textit{Lower bound} = Q1 - 1.5 \times IQR, \quad \textit{Upper bound} = Q3 + 1.5 \times IQR \tag{3.1}$$

Table 3.5: Outlier detection results using the IQR method

Column	IQR	Lower bound	Upper bound	Outliers
min_temp	6	7	31	467
max_temp	6	17	41	550
min_do	2.1	1.75	10.15	811
max_do	1.9	4.25	11.85	587
min_ph	0.5	6.25	8.25	389
max_ph	0.6	7	9.4	160
min_cond	168	-119	553	851
max_cond	600	-599	1,801	969
min_bod	1.41	-1.12	4.53	796
max_bod	3.6	-3.4	11	1,289
min_nitrate	1	-1.3	2.7	260
max_nitrate	3.68	-4.8	9.92	1,757
min_fecal	1,692	-2,530	4,238	663
max_fecal	34,942	-52,355	87,413	2,633

Column	IQR	Lower bound	Upper bound	Outliers
min_tc	2,540	-3,750	6,410	2,289
max_tc	48,720	-72,800	1.22e+5	2,517
min_fs	0	1,276.88	1,276.88	2,736
max_fs	0	57,640.77	57,640.77	2,737

The counts add up to 22,461 flagged values, but one row can contain several outliers. The notebook then removes every row that contains at least one outlier in any of the eighteen columns, using:

```
df = df[~outliers.any(axis=1)].copy()
```

Only 4,051 of the 11,721 rows contain no outlier, so 7,670 rows (65.4%) were removed. This is a strict step. It makes the remaining values more regular, but it also has an important side effect for `min_fs` and `max_fs`. Since most of their entries were replaced by one identical mean value, their IQR is 0 and both quartiles equal that value. Consequently every row with a real, observed value in these columns (2,736 rows) is flagged as an outlier and removed, while the filled rows remain. The surviving data therefore contains almost no genuine fecal streptococci measurements. This is a limitation of the preprocessing, and it is discussed again in Chapter 6. The subsequent models use only the 4,051 retained rows.

3.12 Numeric Columns Analysis

Figure 3.1 is a bar chart of the average `max_bod` for each of the 32 states and union territories, sorted from highest to lowest. It is calculated from the mean-filled data before outlier removal. Delhi has by far the highest average (about 35.0), followed by Tamil Nadu (about 18.8), Gujarat (about 14.0), Uttar Pradesh (about 13.4) and Haryana (about 11.6). The lowest averages are found for the entry named Rd_Bangalore (about 0.4), Kerala (about 1.6) and Tripura (about 2.1). The chart shows that the BOD indicator differs a great deal between regions; it does not by itself explain why, and the number of stations differs from state to state, so the averages should not be read as a complete ranking of river health.

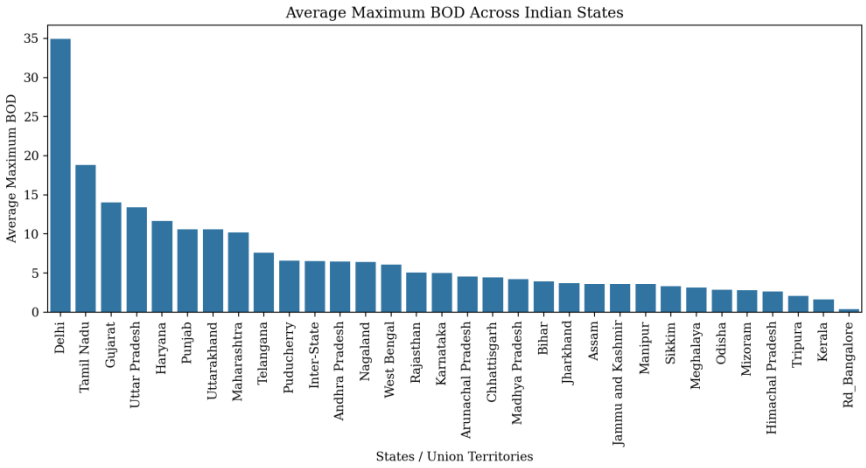


Figure 3.1: Average Maximum BOD Across Indian States

Figure 3.2 shows the boxplot of all eighteen measurement columns drawn on one common axis, as in the notebook. Because the columns have very different scales, the plot is dominated by the columns with the largest values, such as `max_fecal` and `max_tc`, whose extreme points reach billions, while columns with small values appear almost flat near zero. The plot is still useful because it shows at a glance that nearly every column has many points beyond its whiskers, which agrees with the outlier counts of Table 3.5.

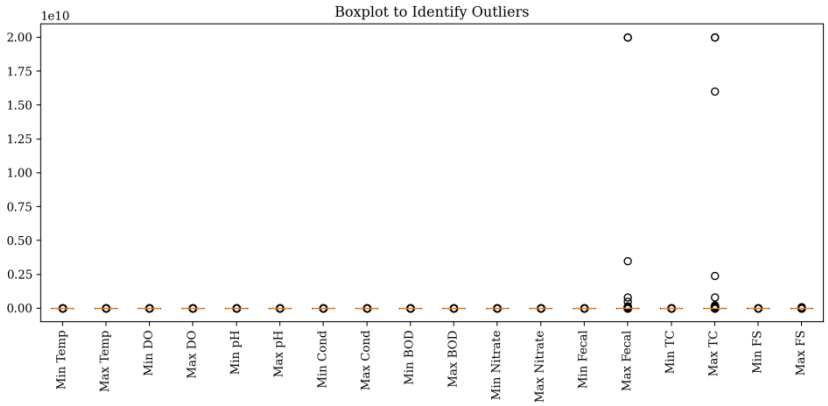


Figure 3.2: Boxplot of all eighteen water-quality columns (outlier identification)

3.13 Correlation Analysis

The correlation matrix of the eighteen columns was computed with `df[water_cols].corr()` and drawn as the heatmap of Figure 3.3. A correlation near +1 means that two columns rise together, a value near −1 means that one falls when the other rises, and a value near 0 means that no straight-line relation is visible. The matrix was computed on the mean-filled data before outliers were removed.

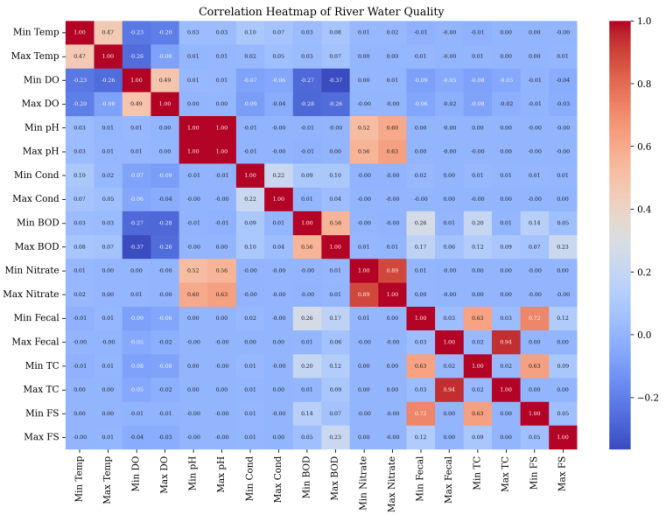


Figure 3.3: Correlation Heatmap of River Water Quality

Several features of the heatmap can be read directly. The strongest relation is between `max_fecal` and `max_tc` (0.94), and `min_fecal` is strongly related to `min_fs` (0.72) and `min_tc` (0.63). The minimum and maximum of the same indicator are moderately related, for example `min_do` and `max_do` (0.49) and `min_temp` and `max_temp` (0.47). For the BOD target of Section 4.1, the strongest relation is with `min_bod` (0.56), and `max_bod` has negative correlations with `min_do` (−0.37) and `max_do` (−0.26), so that records with more dissolved oxygen tend to have lower BOD. Many other pairs are close to zero, for example the pH columns with nearly everything else. The reader should remember that the relations of `min_fs` and `max_fs` are influenced by the mean filling, and that a correlation does not prove a cause.

3.14 Summary of EDA Findings

Table 3.6: Summary of EDA findings and their effect on modelling

Finding	Evidence	Effect on modelling
Many missing values	30,275 in the measurement columns; min_fs and max_fs about 77%	Filled with the column mean; filled values reduce variation
Extreme values in all columns	Table 3.5; boxplot of Figure 3.2	Rows with outliers were removed; 4,051 rows remain
IQR rule fails for fs columns	IQR = 0 after mean filling	Genuine fs measurements removed with the outliers
Strong regional differences in BOD	Figure 3.1	State is not used as a model input
Related columns	Figure 3.3: e.g. max_fecal and max_tc 0.94	Coefficients are read with care
Very different scales	Table 3.2	Scaling is used for Logistic Regression and KNN

4. ANALYSIS ON DATASET

This chapter describes the four models built in the notebook. Each model follows the five parts required by the university format: introduction, general description, specific requirements, functions and formulas, analysis results, and visualization. All four models use the cleaned data of 4,051 rows. Every model uses an 80% training and 20% testing split with `random_state = 42`, which gives 3,240 training rows and 811 test rows. A split is needed because a model must be judged on records it has never seen; results on the training rows alone would only show how well the model remembers them. Fixing the random state makes the same rows fall into each part every time the notebook is run, so the results can be repeated.

4.1 Multiple Linear Regression

4.1.1 Introduction

Multiple Linear Regression (MLR) is the simplest regression model in this project. It predicts a number as a weighted sum of several input features. It is fast, easy to read and gives a baseline for judging more complex models. Here it is used to predict the maximum biochemical oxygen demand (`max_bod`) of a monitoring record from nine other measurements.

4.1.2 General Description

The model assumes that the target changes in a straight-line way with each input when the others are held fixed. The dependent variable is `max_bod`. The nine independent variables are `min_do`, `max_do`, `min_ph`, `max_ph`, `min_cond`, `max_cond`, `min_bod`, `min_nitrate` and `max_nitrate`. They were chosen because oxygen, acidity, conductivity and nutrient measurements are commonly looked at together with BOD, and because `min_bod` of the same record is expected to move with `max_bod`; `max_bod` itself is of course excluded. During training, the model finds the coefficients that make the sum of squared prediction errors as small as possible.

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_9 x_9 \tag{4.1}$$

Here $\hat{y}$ is the predicted `max_bod`, β_0 is the intercept, and β_1 to β_9 are the coefficients of the nine features. The method rests on a few assumptions: the relation is roughly a straight line, the features are not too strongly related to each other, and the data does not contain very

extreme values. The outlier removal of Section 3.11 helps with the last point. The model is evaluated on the 811 test rows that were not used for training.

4.1.3 Specific Requirements, Functions and Formulas

The implementation needs `LinearRegression` and `train_test_split` from Scikit-learn and the metric functions. The notebook code is:

```
features = [min_do, max_do, min_ph, max_ph, min_cond, max_cond,
            min_bod, min_nitrate, max_nitrate]
X = df[features]
y = df[max_bod]
data = pd.concat([X, y], axis=1).dropna()
X = data[features];  y = data[max_bod]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

The code works in four steps. First, the nine feature columns are placed in `X` and the target in `y`, and any row with a missing value is dropped (none are left after cleaning). Second, `train_test_split` divides the rows into training and testing parts. Third, `LinearRegression()` creates an untrained model and `fit` learns the intercept and coefficients from the training rows. Fourth, `predict` applies the learned equation to the test features, and the predictions are compared with the real values `y_test`. The model is evaluated with four metrics, which are explained in Table 4.1.

Table 4.1: Regression metrics and how to read them

Metric	What it measures	Better when
MAE	Average absolute difference between actual and predicted values	Lower
MSE	Average of the squared differences; large errors are punished strongly	Lower
RMSE	Square root of MSE; same unit as the target	Lower
R^2 Score	Share of the variation in the target explained by the model	Closer to 1

$$MAE = (1/n)\ \Sigma\ |y_i - \hat{y}_i|, \qquad MSE = (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2 \tag{4.2}$$

$$RMSE = \sqrt{MSE}, \qquad R^2 = 1 - \Sigma\ (y_i - \hat{y}_i)^2 \,/\, \Sigma\ (y_i - \bar{y})^2 \tag{4.3}$$

4.1.4 Analysis Results

Table 4.2 gives the metrics measured on the 811 test rows.

Table 4.2: Evaluation metrics of Multiple Linear Regression (target: max_bod)

Metric	Value
MAE	0.9442791943126752
MSE	1.8948726384639738
RMSE	1.3765437292232943
R^2 Score	0.4656129077886193

On average the prediction differs from the actual maximum BOD by about 0.94 units (MAE). The RMSE of 1.38 is larger than the MAE, which shows that some predictions have larger errors than the average. To judge the size of the error, it helps to compare it with the spread of the data: in the test set the actual `max_bod` has a mean of about 2.95 and a standard deviation of about 1.88, so an RMSE of 1.38 is clearly smaller than simply guessing the mean every time. The R^2 of 0.4656 means that the model explains about 47% of the variation in `max_bod` on the test data, so more than half of the variation is left unexplained. This is a moderate result, not a strong one.

Table 4.3 lists the coefficients learned by the model. They were obtained by re-running the notebook code, since the notebook does not print them. The coefficient of `min_bod` is by far the largest (about 1.26): when the minimum BOD of a record rises by one unit and the other features stay the same, the predicted maximum BOD rises by about 1.26 units. The coefficient of `min_do` is negative (about −0.43), which agrees with the negative correlation seen in Figure 3.3. The inputs have very different scales and some are related, so the coefficients should not be compared with one another as measures of importance.

Table 4.3: Coefficients of the Multiple Linear Regression model

Feature	Coefficient
min_do	-0.4307
max_do	0.0399
min_ph	0.0227
max_ph	0.2119
min_cond	-0.0025
max_cond	0.0011
min_bod	1.2615
min_nitrate	-0.3085
max_nitrate	0.1473
Intercept	1.4845

Table 4.4 compares actual and predicted values for the first eight rows of the test set. The model is close for some records (for example row 8789 and row 9663) and further away for others (for example row 7220 and row 6610), which is what a model with R^2 below 0.5 would be expected to do.

Table 4.4: Actual and predicted max_bod for the first eight test records (MLR)

Row index	Actual	Predicted
8789	2.9	3.727
767	2.0	2.379
7220	3.7	2.951
7995	2.2	1.597
9663	2.6	2.927
6610	4.0	5.390
8297	5.0	4.544
7211	3.0	4.408

4.1.5 Visualization

Figure 4.1 plots the actual maximum BOD on the x-axis and the predicted value on the y-axis for the test rows. The red line is the ideal line, on which the predicted value equals the actual value; it was added to the notebook scatter plot to make the reading easier. The points form a cloud around the line with a clear spread, and they are not all near it. Points far above or below the line are records for which the model made a large error. This agrees with the moderate R^2 of 0.4656.

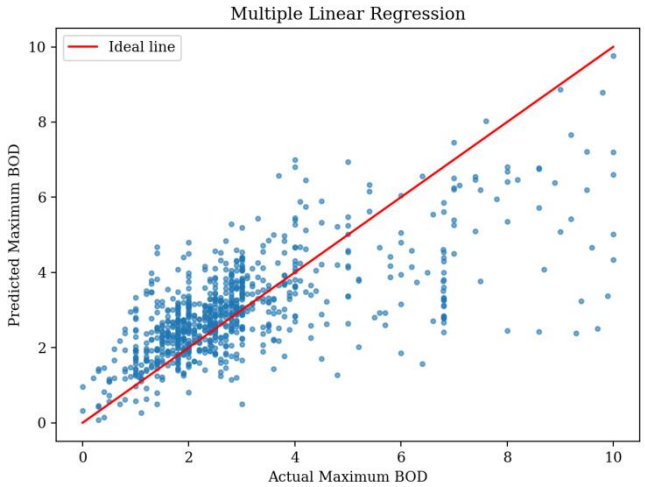


Figure 4.1: Actual vs Predicted Values – Multiple Linear Regression

4.2 Polynomial Regression (Degree = 2)

4.2.1 Introduction

Polynomial Regression extends linear regression so that the fitted line can bend. It is considered when the relation between an input and the target may be curved. In this project it is used in its simplest form: the maximum temperature (`max_temp`) of a record is predicted from the minimum temperature (`min_temp`) of the same record, with a polynomial of degree 2.

4.2.2 General Description

The model does not change the learning method. The input is first transformed into new features, namely a constant term, the original value and its square, and an ordinary linear regression is then fitted on these new features. With one input and degree 2 the transformed data has three columns. The model is still linear in its coefficients, but it can describe a curve.

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 \tag{4.4}$$

Here x is `min_temp` and ŷ is the predicted `max_temp`. The reason for trying degree 2 is that a straight line forces the same rate of change at every temperature, while a curve lets the rate change. A higher degree gives more freedom but also more risk of overfitting, which means fitting the noise of the training rows instead of the real pattern. Performance is therefore measured only on the test rows. Both temperature columns pass through the cleaning steps of Chapter 3 in the same way as the others, and after outlier removal `min_temp` lies between 7 and 31 and `max_temp` between 17 and 41.

4.2.3 Specific Requirements, Functions and Formulas

The implementation uses `PolynomialFeatures` and `LinearRegression`. The transformer is fitted on the training data only, and the same transformation is then applied to the test data:

```
X = df[[min_temp]]
y = df[max temp]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
X_test_poly  = poly.transform(X_test)

model = LinearRegression()
model.fit(X_train_poly, y_train)
y_pred = model.predict(X_test_poly)
```

The difference between `fit_transform` and `transform` is important. `fit_transform` is used on the training data and prepares the transformer; `transform` is used on the test data and only applies the same rules, so that no information from the test rows enters the training process. The transformed training data is then given to an ordinary `LinearRegression` model, exactly as in Section 4.1. The same four metrics (Equations 4.2 and 4.3) are used.

4.2.4 Analysis Results

Table 4.5: Evaluation metrics of Polynomial Regression, degree 2 (target: max_temp)

Metric	Value
MAE	3.018167901033082
MSE	15.183066407526923
RMSE	3.8965454453306356
R^2 Score	0.08531238689290155

The R^2 of 0.0853 means that the model explains only about 8.5% of the variation in `max_temp` on the test data. The RMSE of about 3.90 is only slightly below the standard deviation of the actual test values (about 4.08), which is the error that would be made by always predicting the mean. The MAE of about 3.0 means that the predicted maximum temperature is typically 3 units away from the actual one. These numbers show that `min_temp` alone is a weak predictor of `max_temp` in this dataset, even though the two columns are positively correlated (0.47 in Figure 3.3). The fitted equation, obtained by re-running the notebook, is $\hat{y} = 21.913 + 0.4867x - 0.00547x^2$, where the squared term is very small. Within the data range of 7 to 31 the fitted curve is almost a gentle upward line, so the extra curvature adds little.

Table 4.6 shows the first eight test records. Many predictions lie close to 29 or 30 whatever the actual value is, which is typical of a model that explains little of the variation.

Table 4.6: First eight test records: min_temp, actual and predicted max_temp

Row	min_temp	Actual max_temp	Predicted max_temp
1	17.0	30.0	28.61
2	21.2	24.6	29.77
3	17.0	31.0	28.61
4	30.0	32.0	31.59
5	16.0	26.0	28.30
6	24.0	35.0	30.44
7	21.0	27.0	29.72
8	24.0	27.0	30.44

4.2.5 Visualization

Figure 4.2 shows the test values of `min_temp` on the x-axis, with the actual `max_temp` in one colour and the predicted `max_temp` in another. The predicted points lie on a smooth, nearly flat curve, while the actual points are spread widely above and below it at every value of `min_temp`. This picture matches the low R^2 of 0.0853: one input is not enough to place the actual points near the curve.

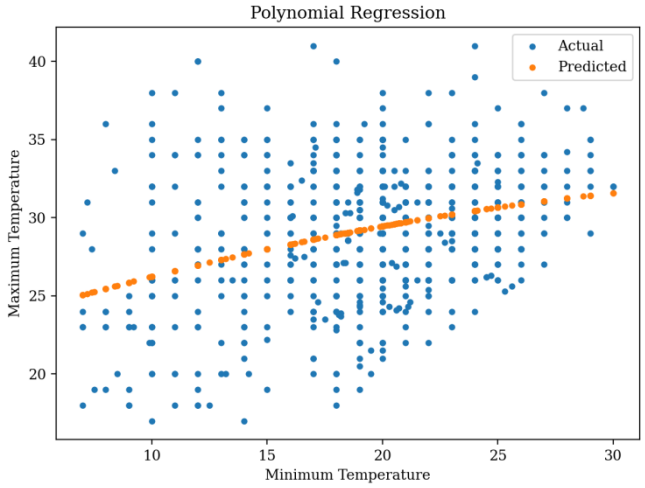


Figure 4.2: Actual vs Predicted Values – Polynomial Regression (degree 2)

4.3 Logistic Regression

4.3.1 Introduction

Classification means predicting a category instead of a number. Logistic Regression is a basic and widely used classification method. In this project it is used to decide whether the maximum fecal coliform (`max_fecal`) of a record is high or low, using twelve physical and chemical measurements as inputs.

4.3.2 General Description

The notebook first creates the class label. The median of `max_fecal` in the cleaned data is 310, and a record gets class 1 (“high”) if its `max_fecal` is at least 310 and class 0 (“low”) otherwise. This gives 50.04% of the records in class 1, so the two classes are almost equal in size. The twelve inputs are `min_temp`, `max_temp`, `min_do`, `max_do`, `min_ph`, `max_ph`, `min_cond`, `max_cond`, `min_bod`, `max_bod`, `min_nitrate` and `max_nitrate`. The fecal, total coliform and streptococci columns are not used as inputs.

Logistic Regression computes a weighted sum z of the features, as in linear regression, and passes it through the sigmoid function, which converts it into a probability between 0 and 1. By default a record is assigned class 1 if this probability is above 0.5.

$$p = 1 / (1 + e^{-z}), \qquad z = \beta_0 + \beta_1 x_1 + \ldots + \beta_{12} x_{12} \tag{4.5}$$

The word “regression” in the name can be confusing, because the method is used for classification. It is called regression because it first calculates a number z in the same way as linear regression; the sigmoid function and the 0.5 cut-off then turn z into a class. The choice of a median split has a practical meaning: it asks whether a record is above or below the typical contamination level of this particular dataset. It does not use any official water-quality limit.

4.3.3 Specific Requirements, Functions and Formulas

The model needs `LogisticRegression`, `StandardScaler` and the classification metrics. The notebook scales the features before training so that all inputs have a similar range, which helps the solver to converge. The scaler learns the mean and standard deviation from the training rows only:

```
threshold = y_value.median()
y = (y_value >= threshold).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

accuracy_score(y_test, y_pred)
confusion_matrix(y_test, y_pred)
```

The notebook uses `predict` to obtain the classes and does not call `predict_proba`. Scikit-learn can return the class probability with `predict_proba`, and this is the value that is compared with 0.5 inside `predict`. The evaluation uses accuracy and the confusion matrix. Accuracy is the share of correct predictions. The confusion matrix counts true negatives (TN), false positives (FP), false negatives (FN) and true positives (TP). From these counts, precision, recall and the F1-score can be calculated for each class.

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \tag{4.6}$$

$$Precision = TP / (TP + FP), \quad Recall = TP / (TP + FN) \tag{4.7}$$

$$F1 = 2 \times Precision \times Recall / (Precision + Recall) \tag{4.8}$$

4.3.4 Analysis Results

The accuracy on the 811 test rows is 73.74% (0.7373612823674476). Table 4.7 shows the confusion matrix. The test set contains 406 records of class 0 and 405 of class 1.

Table 4.7: Confusion matrix of Logistic Regression (max_fecal)

Actual \ Predicted	Predicted 0 (low)	Predicted 1 (high)
Actual 0 (low)	294	112
Actual 1 (high)	101	304

Of the 406 low records, 294 were found (recall 0.72) and 112 were wrongly predicted as high. Of the 405 high records, 304 were found (recall 0.75) and 101 were wrongly predicted as low. The two kinds of error are close in size, so the model does not favour one class strongly. Table 4.8 gives the class-wise measures calculated from the confusion matrix; the notebook itself prints only the accuracy and the matrix.

Table 4.8: Class-wise precision, recall and F1-score of Logistic Regression (calculated from Table 4.7)

Class	Precision	Recall	F1-score	Support
0 (low max_fecal)	0.74	0.72	0.73	406
1 (high max_fecal)	0.73	0.75	0.74	405
Accuracy			0.74	811
Macro average	0.74	0.74	0.74	811

Because the classes are almost balanced (50.04% against 49.96%), a model that always predicted the larger class would score only about 50%, so the accuracy of 73.74% is a real improvement over guessing. Accuracy alone, however, still does not tell the whole story. It

does not show which kind of error is made or whether one class is handled worse than the other, which is why the confusion matrix and class-wise recall are also reported. About one prediction in four is wrong, so the model is useful as a rough separator of high and low records but cannot be trusted for individual records. It should also be remembered that the labels come from a median split: records close to the threshold of 310 are separated only by a very small difference in the original value, and these records are naturally the hardest to classify.

Table 4.9 lists the coefficients of the model on the scaled inputs, obtained by re-running the notebook. A positive value pushes the prediction towards the "high max_fecal" class. The largest positive coefficients belong to `max_do` (0.79), `max_temp` (0.58) and `min_bod` (0.38), and the largest negative to `min_do` (−0.42) and `min_cond` (−0.34). Since `min_do` and `max_do` are related columns with opposite signs, individual coefficients should not be read as separate effects.

Table 4.9: Coefficients of the Logistic Regression model on scaled inputs

Feature (scaled)	Coefficient	Feature (scaled)	Coefficient
min_temp	0.1347	max_cond	-0.3181
max_temp	0.5773	min_bod	0.3847
min_do	-0.4227	max_bod	-0.2202
max_do	0.7939	min_nitrate	-0.2795
min_ph	-0.1433	max_nitrate	-0.2188
max_ph	-0.0591	Intercept	0.0175
min_cond	-0.3381		

4.3.5 Visualization

Figure 4.3 shows the confusion matrix as a heatmap. The two dark-blue cells on the diagonal hold the correct predictions (294 and 304), and the lighter off-diagonal cells hold the errors (112 and 101). A good classifier shows large values on the diagonal and small values elsewhere; here the diagonal is clearly larger but the errors are not small.

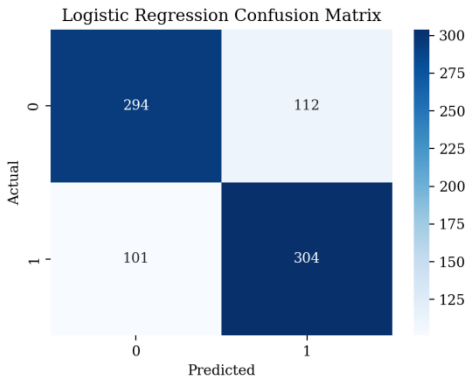


Figure 4.3: Confusion Matrix – Logistic Regression

4.4 K-Nearest Neighbors (KNN) Classification

4.4.1 Introduction

K-Nearest Neighbors is a simple classification method that does not fit a formula. It stores the training records and decides the class of a new record by looking at the most similar stored records. In this project it is used to decide whether the maximum dissolved oxygen (`max_do`) of a record is high or low.

4.4.2 General Description

The class label is created in the same way as for Logistic Regression. The median of `max_do` in the cleaned data is 8.0; a record gets class 1 if `max_do` is at least 8.0 and class 0 otherwise, which gives 52.78% in class 1. The eleven inputs are `min_temp`, `max_temp`, `min_ph`, `max_ph`, `min_cond`, `max_cond`, `min_bod`, `max_bod`, `min_nitrate`, `max_nitrate` and `max_fecal`. The two dissolved oxygen columns are not used as inputs, since `max_do` is the target.

For each test record, KNN measures the distance to every training record in the space of the eleven features, picks the K closest records, and gives the class chosen by most of them. Here K = 5. This value is small, it is odd so that the vote between two classes cannot end in a tie, and it is the default value of Scikit-learn; the notebook did not test other values. Because the method relies on distance, features with large numbers, such as `max_cond` or `max_fecal`, would control the distance and hide features with small numbers, such as `max_ph`. For this reason the features are scaled with `StandardScaler` before using KNN.

$$d(x, x') = \sqrt{\left(\Sigma_{j=1}^{11} (x_j - x'_j)^2 \right)}, \qquad z = (x - \mu) / \sigma \tag{4.9}$$

KNN differs from Logistic Regression in a basic way. Logistic Regression learns a small set of coefficients and then no longer needs the training rows. KNN learns nothing in advance; it keeps all 3,240 training rows and uses them each time a prediction is made. For this reason it is called an instance-based method. It does not assume any particular shape for the relation, so it can follow irregular patterns, but it needs more memory and time when predicting.

4.4.3 Specific Requirements, Functions and Formulas

After StandardScaler, every feature has a mean of about 0 and a standard deviation of about 1, so a difference of one unit means a similar thing for every feature. The scaler learns the mean μ and the standard deviation σ from the training rows only and applies them to both parts, so that no information from the test rows is used. The code is:

```
threshold = y_value.median()
y = (y value >= threshold).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
```

The classifier is created with `n_neighbors=5`, fitted on the scaled training data and used to predict the classes of the scaled test data. The same metrics as for Logistic Regression (Equations 4.6 to 4.8) are used, so that the two classifiers can be compared directly. Only the predicted classes are evaluated; probabilities were not requested from KNN in the notebook.

4.4.4 Analysis Results

The accuracy on the 811 test rows is 76.08% (0.7607891491985204). Table 4.10 shows the confusion matrix. The test set contains 361 records of class 0 and 450 of class 1.

Table 4.10: Confusion matrix of the KNN classifier (max_do)

Actual \ Predicted	Predicted 0 (low)	Predicted 1 (high)
Actual 0 (low)	276	85
Actual 1 (high)	109	341

KNN found 276 of the 361 low-dissolved-oxygen records (recall 0.76) and 341 of the 450 high records (recall 0.76). It made 85 errors of the first kind and 109 of the second kind. The

errors are again of similar size for both classes, and the recall values for the two classes are almost equal. Table 4.11 gives the class-wise measures calculated from the confusion matrix.

Table 4.11: Class-wise precision, recall and F1-score of KNN (calculated from Table 4.10)

Class	Precision	Recall	F1-score	Support
0 (low max_do)	0.72	0.76	0.74	361
1 (high max_do)	0.80	0.76	0.78	450
Accuracy			0.76	811
Macro average	0.76	0.76	0.76	811

A model that always predicted class 1 would reach about 55.5% on this test set (450 of 811), so the accuracy of 76.08% is a clear gain. The precision of class 1 (0.80) is higher than that of class 0 (0.72), which means that when KNN says “high dissolved oxygen” it is right four times in five. Three points limit the result. K = 5 was used without testing other values, a larger K would give smoother decisions and a smaller K would follow local detail more closely, so the result could change. The prediction uses the maximum fecal coliform as one of its inputs, which makes the problem different from the one solved by Logistic Regression. And a median split again places many records close to the threshold, where errors are most likely.

Table 4.12: Logistic Regression and KNN: approach and result

Aspect	Logistic Regression	KNN (K = 5)
Target	max_fecal (threshold 310)	max_do (threshold 8.0)
Number of inputs	12	11
Learning style	Learns coefficients	Stores the training rows
Scaling used	Yes (StandardScaler)	Yes (StandardScaler)
Correct predictions	598 of 811	617 of 811
Accuracy	73.74%	76.08%

4.4.5 Visualization

Figure 4.4 shows the confusion matrix of KNN. The green cells on the diagonal hold the correct predictions (276 and 341) and the off-diagonal cells hold the errors (85 and 109). Compared with Figure 4.3, the diagonal values are balanced between the classes in the same way, and the number of correct predictions is higher in total.

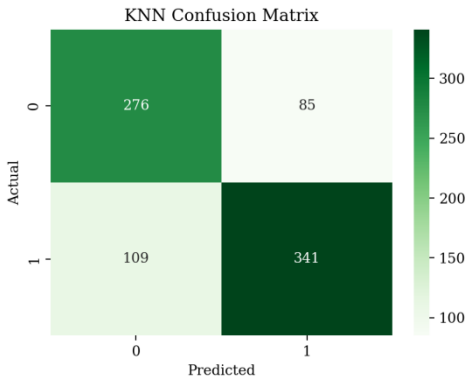


Figure 4.4: Confusion Matrix – KNN Classifier

4.5 Comparative Analysis of Regression Models

Table 4.13: Comparison of regression models

Model	MAE	MSE	RMSE	R^2
Multiple Linear Regression (max_bod)	0.9443	1.8949	1.3765	0.4656
Polynomial Regression (max_temp)	3.0182	15.1831	3.8965	0.0853

According to the evaluated metrics, Multiple Linear Regression performs better: it has the lower MAE, MSE and RMSE and the higher R^2 (0.4656 against 0.0853). One point must be stated clearly. The two models do not predict the same target. MLR predicts `max_bod` from nine inputs, and Polynomial Regression predicts `max_temp` from a single input. The errors (MAE, MSE, RMSE) are in the units of different columns and cannot be compared directly; only R^2, which is a relative measure, can be placed side by side, and even then it compares two different problems. The correct reading is that nine inputs explain nearly half of the variation in BOD, while `min_temp` alone explains less than a tenth of the variation in `max_temp`. Polynomial Regression is therefore not an improvement over MLR in this project, and its degree-2 terms add very little to what a simple straight line would give.

4.6 Comparative Analysis of Classification Models

Table 4.14: Comparison of classification models

Model	Accuracy	Recall (class 0)	Recall (class 1)	Macro F1	Errors (FP + FN)
Logistic Regression	73.74%	0.72	0.75	0.74	213
KNN (K = 5)	76.08%	0.76	0.76	0.76	194

The classifiers are compared on two objectives. The first is overall accuracy: KNN is higher by about 2.3 percentage points (76.08% against 73.74%). The second is balance between the classes, measured by the recall of each class and by the macro F1-score: KNN again has equal or higher values (recall 0.76 for both classes, macro F1 0.76) than Logistic Regression (recall 0.72 and 0.75, macro F1 0.74). On both objectives KNN has a small advantage. The comparison needs care for the same reason as before: the two models solve different problems (`max_fecal` against `max_do`) with different input sets and different thresholds, so the difference does not prove that KNN is the better algorithm. Both models used scaled inputs and one random split, and a difference of two percentage points on 811 test records could change with another split.

4.7 Overall Model Performance Discussion

Table 4.15: Overall comparison of the four models

Task	Model	Main metric	Remark
Regression (max_bod)	Multiple Linear Regression	$R^2 = 0.4656$	RMSE 1.3765; nine inputs
Regression (max_temp)	Polynomial Regression	$R^2 = 0.0853$	RMSE 3.8965; one input
Classification (max_fecal)	Logistic Regression	Accuracy = 73.74%	Recall 0.72 / 0.75
Classification (max_do)	KNN (K = 5)	Accuracy = 76.08%	Recall 0.76 / 0.76

Overall, the models show moderate performance. The best regression result explains about 47% of the variation of its target, and the classifiers are right about three times in four. No model is outstanding. Several limits should be kept in mind: the results come from one random split without cross-validation; the preprocessing removed about 65% of the rows and effectively removed the genuine fecal streptococci values; no hyperparameters were tuned; and the four models use different targets and inputs. Small differences should therefore not be treated as firm proof that one model is better than another.

5. CONCLUSION

5.1 Project Summary

This project used a river water-quality dataset covering monitoring stations in 32 Indian states and union territories to study the relations between the measured indicators and to build four prediction models. The work covered data inspection, missing-value handling, outlier detection and removal, visual exploration, model building with four algorithms, evaluation and comparison. All results in this report come from the code and the executed outputs of `project.py`.

5.2 Major Activities and Findings

The original data had 11,721 rows and 23 columns, 30,278 missing values and no duplicate rows. The missing values of the eighteen measurement columns were filled with column means. The IQR rule flagged 22,461 values, and the removal of every row containing an outlier left 4,051 rows. The exploratory charts showed large differences in average BOD between states (highest for Delhi), very wide ranges in almost every column, and strong relations such as that between `max_fecal` and `max_tc` (0.94). It was also found that the mean filling of the very incomplete `min_fs` and `max_fs` columns makes the IQR rule remove nearly all of their genuine values.

5.3 Model Results

For regression, Multiple Linear Regression predicted `max_bod` with an R^2 of 0.4656 and an RMSE of 1.3765, so it explains about 47% of the variation of the target. Polynomial Regression of degree 2 predicted `max_temp` from `min_temp` with an R^2 of 0.0853 and an RMSE of 3.8965, so a single temperature input explains less than a tenth of the variation. For classification, Logistic Regression reached an accuracy of 73.74% for high or low `max_fecal`, and KNN with K = 5 reached 76.08% for high or low `max_do`. In both cases the two classes are almost equal in size and both are predicted with similar recall (0.72 and 0.75 for Logistic Regression, 0.76 and 0.76 for KNN). KNN is slightly higher on accuracy and on class balance, but because the two models solve different problems, the difference is not firm proof that KNN is the better algorithm.

Table 5.1: Summary of key results

Model	Target	Main result	Other result
Multiple Linear Regression	max_bod	$R^2 = 0.4656$	RMSE = 1.3765
Polynomial Regression (degree 2)	max_temp	$R^2 = 0.0853$	RMSE = 3.8965
Logistic Regression	max_fecal (high/low)	Accuracy = 73.74%	Recall 0.72 / 0.75
KNN (K = 5)	max_do (high/low)	Accuracy = 76.08%	Recall 0.76 / 0.76

5.4 Challenges and Learning

The main challenges were the large share of missing data, the extreme and sometimes impossible values, the very different scales of the columns, and the fact that simple general preprocessing steps can have unwanted side effects. The project showed that a model should be judged with several measures and not by one number, that a confusion matrix shows which errors a classifier makes, that scaling is needed for distance-based methods such as KNN, that a more flexible model (the degree-2 polynomial) does not automatically perform better, and that two models can be compared fairly only when they solve the same problem. It also gave practical experience with the full workflow in Python and Scikit-learn.

Table 5.2: Objectives and outcomes

Objective	Outcome	Where
Load and inspect the dataset	Completed; 11,721 rows, 23 columns	Chapters 2–3
Handle missing values, detect and remove outliers	Completed; means filled, 7,670 rows removed	Sections 3.10–3.11
Explore with plots	Completed; bar chart, heatmap, boxplot	Sections 3.12–3.13
Predict max_bod and max_temp	Completed; two regression models	Sections 4.1–4.2
Classify max_fecal and max_do	Completed; two classifiers	Sections 4.3–4.4
Evaluate and compare	Completed; metrics and comparison tables	Sections 4.5–4.7

5.5 Overall Outcome

The project achieved its objectives: four models were built, evaluated with suitable metrics and compared honestly. The results are moderate and should be seen as a learning baseline, not as a final prediction system for river water quality.

6. FUTURE SCOPE

6.1 Completed Work and Proposed Work

The work described in Chapters 3 to 5 was completed in the notebook. Everything listed in this chapter is proposed future work and was not done in this project.

6.2 Modelling Improvements

- Test more regression and classification algorithms, such as decision trees or ensemble methods, and compare them with the present four on the same targets.
- Tune hyperparameters, for example the value of K in KNN and the degree of the polynomial, and use cross-validation to get more reliable scores than a single split.
- Compare a degree-1 and a degree-2 model on the same target to see whether the polynomial terms help at all.
- Improve generalization by comparing training and test scores to detect overfitting.

6.3 Better Handling of Missing Data and Outliers

The mean filling and the removal of every row containing an outlier removed about 65% of the records. A future study can fill missing values with the median or with values from the same state or station, treat the columns `min_fs` and `max_fs` separately because most of their entries are missing, correct or remove impossible values such as pH above 14, and compare the models with and without outlier removal.

6.4 Classification Targets and Metrics

The high and low classes were created from the median of the data. Future work can use official water-quality limits to define the classes, which would make the result easier to interpret. Further metrics, such as the ROC curve, the precision-recall curve and the area under them, can be reported next to accuracy and the confusion matrix. If a future target is imbalanced, class weights, resampling or a changed decision threshold can be applied.

6.5 Features and Data

Features such as the state, the monitoring location and the year were not used as inputs and can be tested, as can the columns left out of each model. Feature selection can be improved by removing strongly related features, for example `max_fecal` and `max_tc`. A time-based

study across the years, and additional data from other sources, could improve prediction and generalization.

6.6 Application Development

A simple prediction interface, in which a user enters water-quality measurements and receives an estimated BOD value or a high/low classification, could be developed. If the models become more reliable, this could later be deployed as a river water quality prediction application.

6.7 Limitations of the Present Work

Table 6.1: Limitations of this project and proposed improvements

Limitation	Seen in	Proposed improvement
Single train-test split	All four models	Cross-validation
Mean filling of missing values	All models; min_fs and max_fs	Median or group-wise filling; separate treatment
About 65% of rows removed as outliers	All four models	Compare with and without removal
K not tuned	KNN	Test several values of K
Different targets and inputs	Model comparison	Compare models on the same target
Median-based class labels	Logistic Regression and KNN	Use official limits to define classes
Single input for polynomial model	Polynomial Regression	Use more inputs

7. REFERENCES

[1] Python Software Foundation, “Python 3 Documentation.” [Online]. Available: https://docs.python.org/3/

[2] The pandas development team, “pandas documentation.” [Online]. Available: https://pandas.pydata.org/docs/

[3] NumPy Developers, “NumPy documentation.” [Online]. Available: https://numpy.org/doc/

[4] Matplotlib Development Team, “Matplotlib documentation.” [Online]. Available: https://matplotlib.org/stable/

[5] M. Waskom, “seaborn: statistical data visualization.” [Online]. Available: https://seaborn.pydata.org/

[6] Scikit-learn Developers, “Linear Models (Linear Regression, Polynomial Regression, Logistic Regression).” [Online]. Available: https://scikit-learn.org/stable/modules/linear_model.html

[7] Scikit-learn Developers, “Nearest Neighbors (KNeighborsClassifier).” [Online]. Available: https://scikit-learn.org/stable/modules/neighbors.html

[8] Scikit-learn Developers, “Preprocessing data (StandardScaler, PolynomialFeatures).” [Online]. Available: https://scikit-learn.org/stable/modules/preprocessing.html

[9] Scikit-learn Developers, “Model evaluation: quantifying the quality of predictions.” [Online]. Available: https://scikit-learn.org/stable/modules/model_evaluation.html

[10] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.